최원준

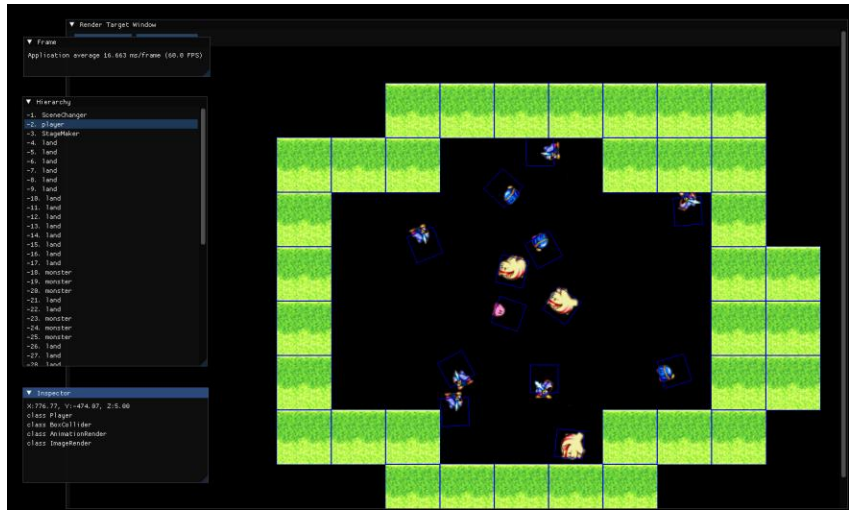
Game Developer Portfolio

목차

1. DirectX9 기반 2D게임 제작 툴 프로젝트 – Kirby
[C++, Win32API, DirectX9, Box2D, IMGUI, FBXSDK]

- 프로젝트 구조
- 주요 기능 요약
- 기능 구현 상세

https://github.com/RuinaR/API_Project-DirextX



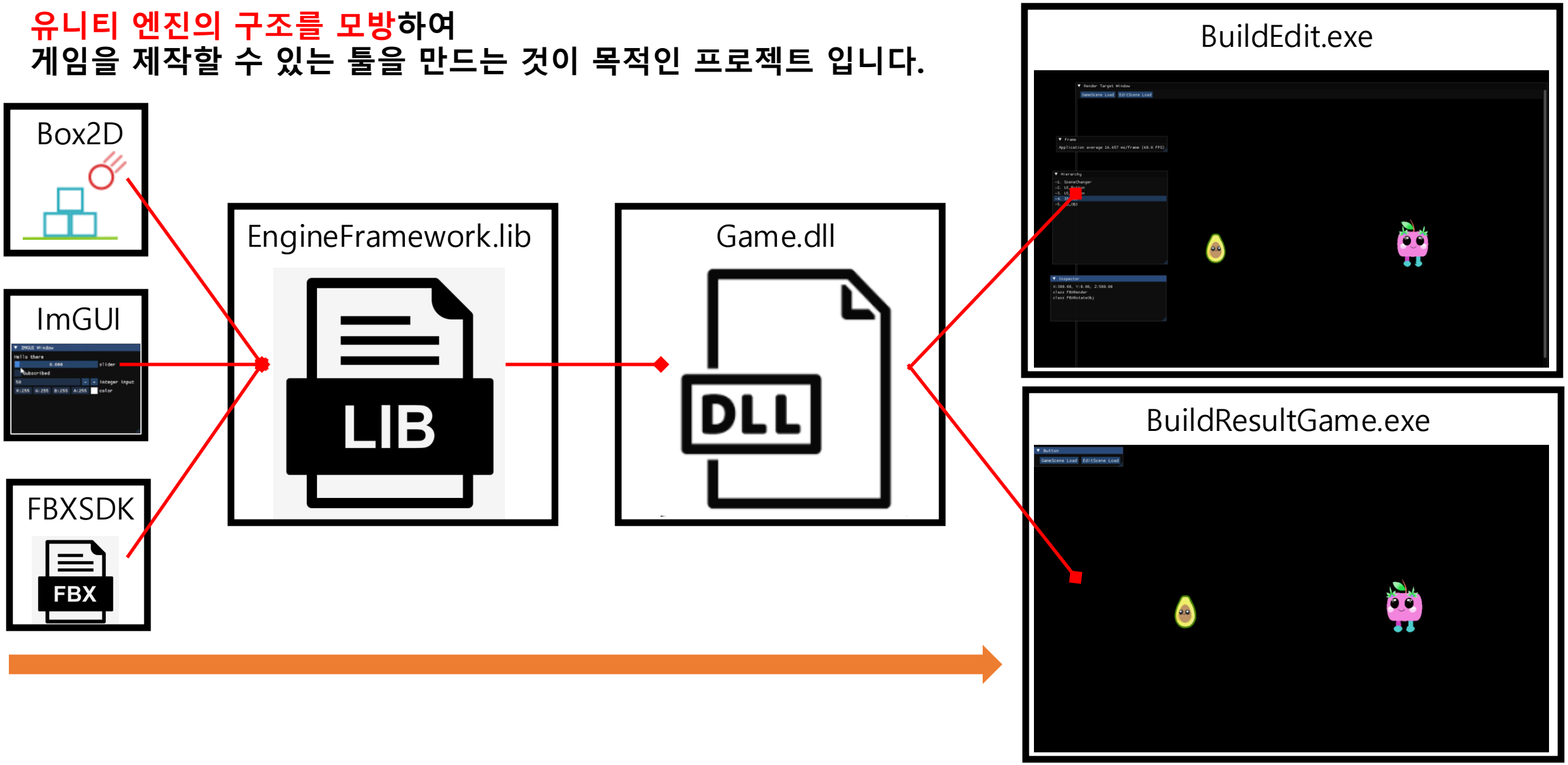
2. Unity 모바일 게임 프로젝트 – Starlight
[C#, Unity]

- 게임 소개
- 게임 시스템

https://github.com/RuinaR/Unity_ClickerGame_StarLight

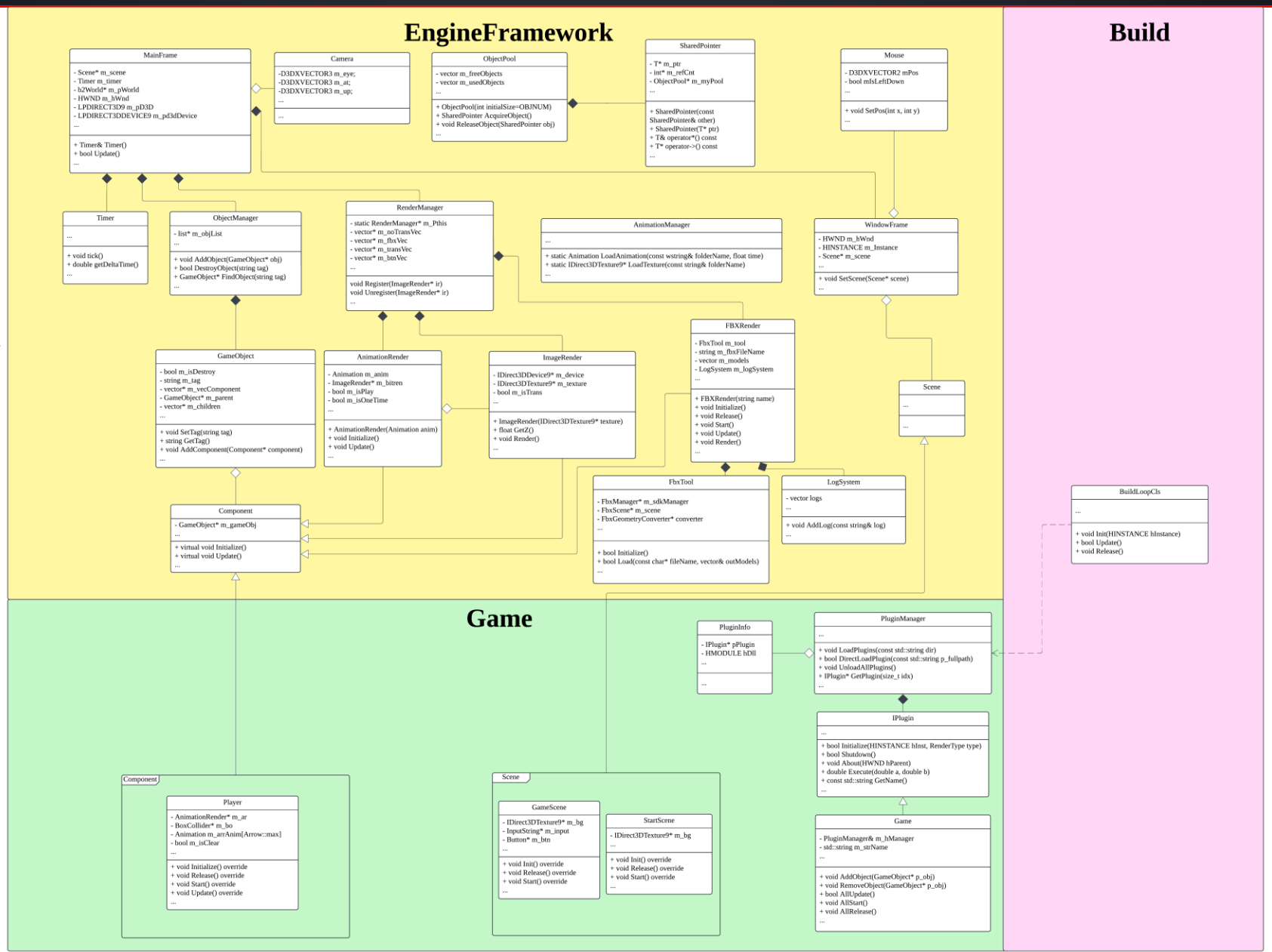
Kirby 프로젝트 구조

유니티 엔진의 구조를 모방하여
게임을 제작할 수 있는 툴을 만드는 것이 목적인 프로젝트 입니다.



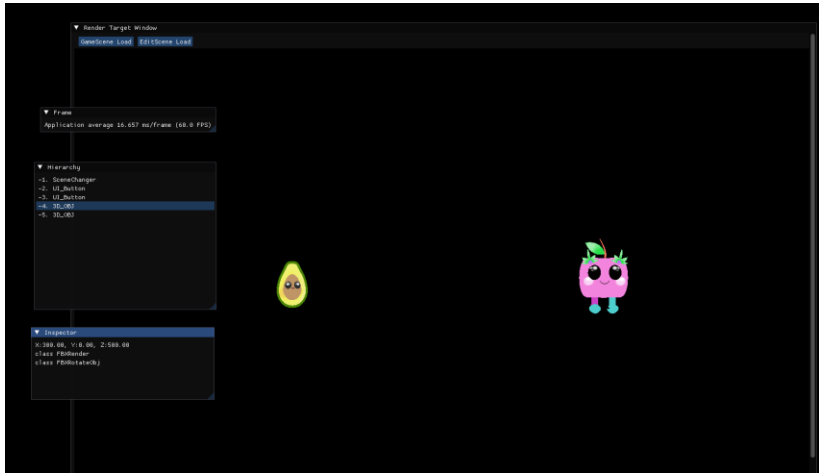
Kirby 프로젝트 구조

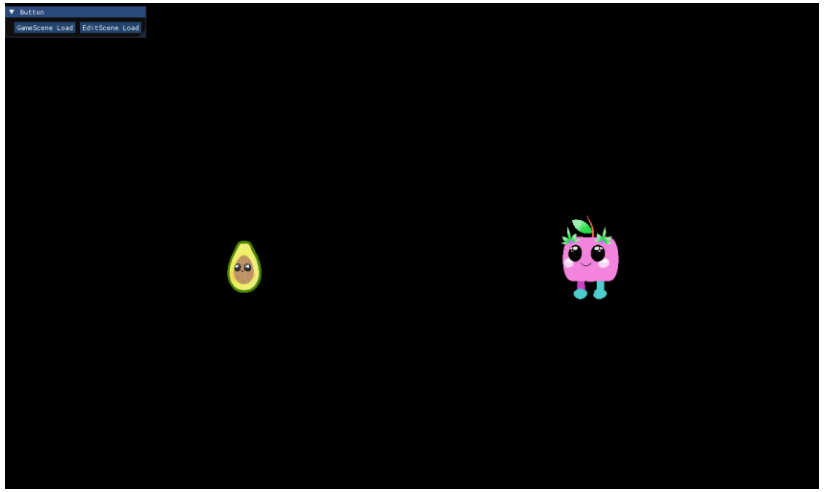
EngineFramework
↓
Game
↓
Build(Edit, Result)



Kirby 프로젝트 주요 기능 요약

1. 컴포넌트 패턴을 활용한 기능 모듈 조합 방식
2. 오브젝트 매니저를 통한 객체 관리 시스템
3. 스마트 포인터(Shared Pointer)와 오브젝트 풀링
4. DirectX9기반의 렌더링 최적화
5. 직접 구현한 2D 물리 처리 시스템 최적화
6. Box2D 라이브러리를 활용한 2D 물리 시뮬레이션
7. ImGUI 라이브러리를 활용한 게임 툴 UI
8. FBXSDK를 활용한 FBX파싱과 렌더링 적용
9. 텍스처 리소스 관리 시스템



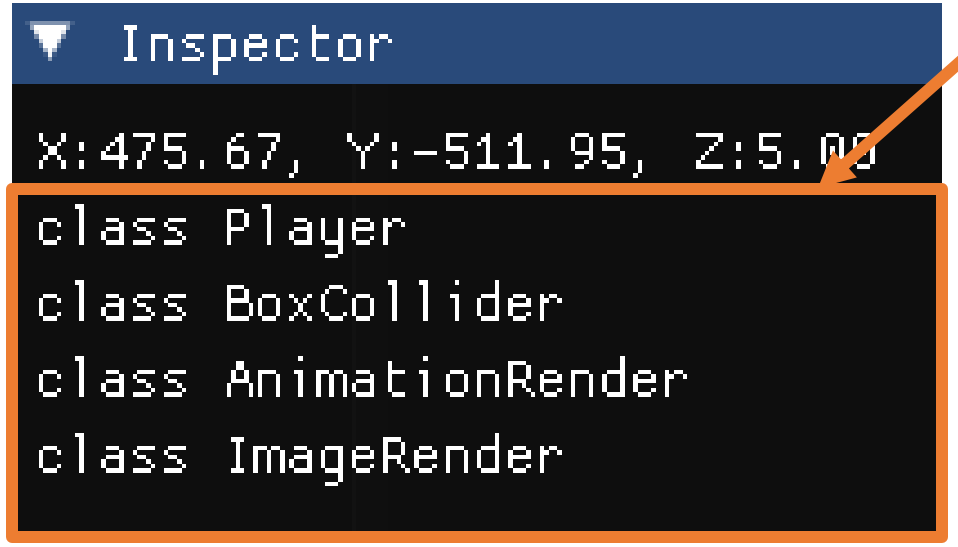


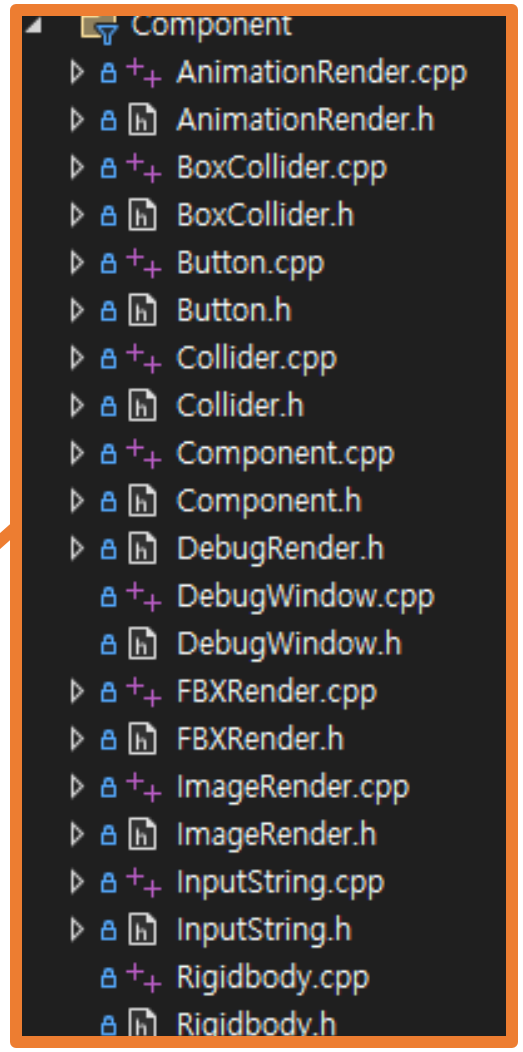
Kirby 프로젝트 기능 구현 상세

1. 컴포넌트 패턴을 활용한 기능 모듈 조합 방식

게임 오브젝트에 **컴포넌트 패턴**을 적용,
기능별로 모듈화 시켜
조합하는 방식으로 사용합니다.

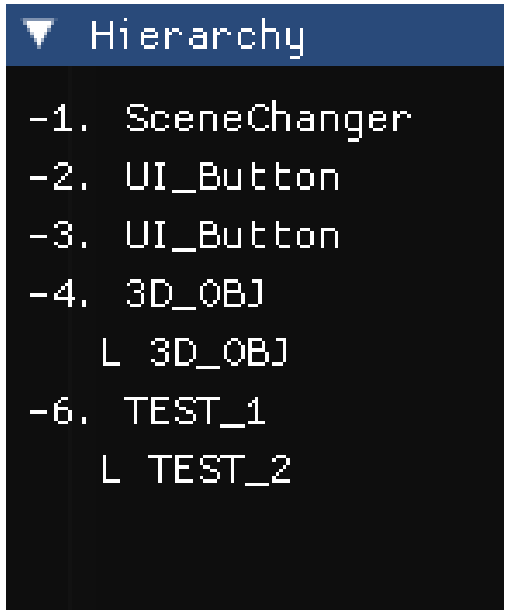
이를 통해 확장성과 재사용성을
크게 높였습니다.






```
class Component abstract
{
protected:
    GameObject* m_gameObj = nullptr;

    virtual void CollisionEnter(Collider* other) {};
    virtual void CollisionExit(Collider* other) {};
    virtual void Collision(Collider* other) {};

public:
    Component();
    virtual ~Component();
    virtual void Initialize() abstract;
    virtual void Release() abstract;
    virtual void Start() abstract;
    virtual void Update() abstract;
    void OnCollisionEnter(Collider* other) { CollisionEnter(other); }
    void OnCollisionExit(Collider* other) { CollisionExit(other); }
    void OnCollision(Collider* other) { Collision(other); }
    void InitGameObj(GameObject* obj);
    GameObject* GetGameObject();

    virtual void OnLBtnDown();
    virtual void OnLBtnUp();
    virtual void OnRBtnDown();
    virtual void OnRBtnUp();
    virtual void OnMouseHover();
};
```

2. 오브젝트 매니저를 통한 객체 관리 시스템




```
//UPDATE
m_pWorld->Step(m_timer.getTotalDeltaTime(), m_velocityIterations, m_positionIterations);
ObjectManager::GetInstance()->Update();

//RENDER
if (m_type == RenderType::Edit)
    RenderManager::GetInstance()->EditUpdate();
else if (m_type == RenderType::Game)
    RenderManager::GetInstance()->GameUpdate();

m_timer.resetTotalDeltaTime(); // 업데이트, 랜더 후 토탈 델타 타임 리셋
```

모든 게임 오브젝트는 생성 시 스스로를
오브젝트 매니저에 등록시키며,

오브젝트 매니저는 현재 Scene에 따라
게임오브젝트들을 일괄 처리합니다.

우측 이미지는 모든 게임오브젝트를
일괄 Update 처리하는 부분입니다.

특정 게임오브젝트를 임의로 삭제할 경우,
즉시 삭제되지 않고 Destroy상태로 전환 한 뒤,
해당 객체의 다음 업데이트 후에 삭제됩니다.

3-1.스마트 포인터(Shared Pointer)

```
template<typename T>
class SharedPointer
{
private:
    T* m_ptr;
    int* m_refCnt;
    ObjectPool<T>* m_myPool;
```

Shared Pointer는C++에서 제공하는
std::shared_ptr를 직접 구현한 것입니다.

```
void release()
{
    if (!m_myPool && m_refCnt && --(*m_refCnt) == 0)
    {
        delete m_ptr;
        delete m_refCnt;
        m_ptr = nullptr;
        m_refCnt = nullptr;
        m_myPool = nullptr;
    }
    else if (m_myPool && m_refCnt && (*m_refCnt) == 1)
    {
        m_myPool->ReleaseObject(this);
    }
}
```

상황에 따라 ObjPool을 통해 사용할 수도 있고,

따로 ObjPool을 등록하지 않고
독자적으로 사용할 수도 있습니다.

3-2. 오브젝트 풀링

```
template <typename T>
class ObjectPool
{
private:
    std::vector<SharedPointer<T>> m_freeObjects; // 사용되지 않는 오브젝트
    std::vector<SharedPointer<T>> m_usedObjects; // 사용 중인 오브젝트
```

```
// 오브젝트 할당 함수
SharedPointer<T> AcquireObject()
{
    if (m_freeObjects.empty())
    {
        // 사용 가능한 오브젝트가 없으면 새 오브젝트 생성
        SharedPointer<T> obj(new T());
        obj.SetPool(this);
        m_usedObjects.push_back(obj);
        return obj;
    }
    else
    {
        // 사용 가능한 오브젝트가 있으면 반환
        SharedPointer<T> obj = m_freeObjects.back();
        m_freeObjects.pop_back();
        m_usedObjects.push_back(obj);
        return obj;
    }
}
```

ObjPool은
사용되지 않는 오브젝트와 사용 중인 오브젝트를
각각 따로 관리합니다.

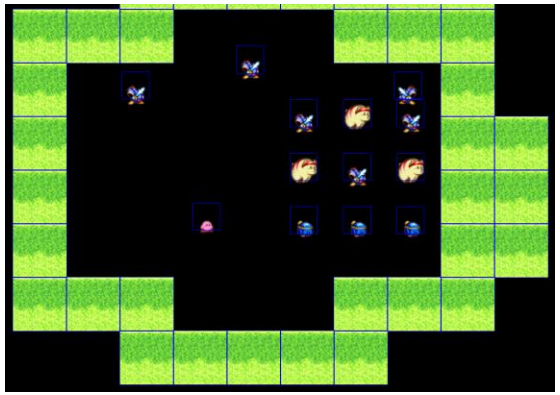
사용자가 ObjPool에 객체를 요청하면,
상황에 따라 사용되지 않는 객체를 꺼내 주거나,
새 객체를 만들어 줍니다.

4. DirectX9기반의 렌더링 최적화

RenderManager에서는 같은 쉐이더 설정을 공유하는 객체끼리 그룹화하여 각 그룹을 따로 그립니다.
해당 방식으로 **드로우 콜을 최소화**하였습니다.

투명, 불투명 객체들은 정확한 렌더링을 위해 카메라 트랜스폼을 기반으로, **Z깊이를 기준으로 각각 반대로 정렬**하여 렌더링합니다.

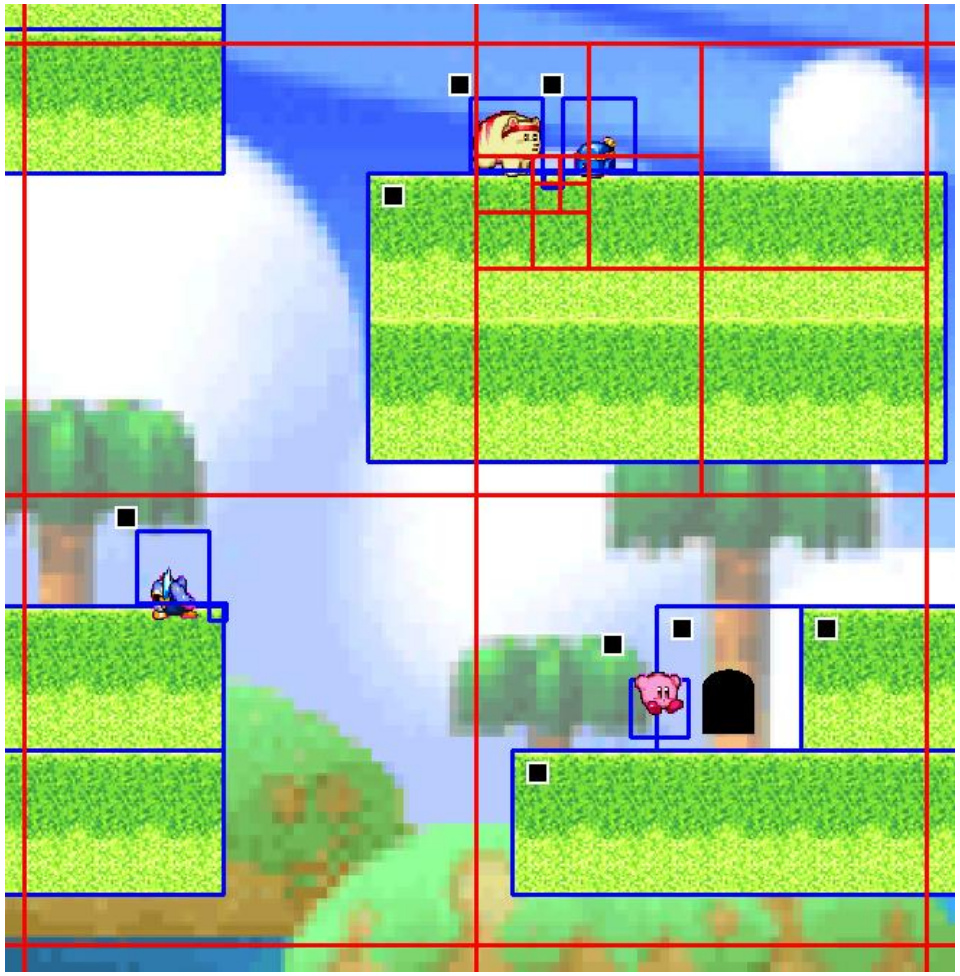
또한 디버그 관련 렌더링 속성을 추가하여, 개발 중 **필요한 정보를 시각적으로 확인**할 수 있도록 하였습니다.



(FBX모델 렌더링 또한 담당합니다.)

```
class RenderManager
{
private:
    static RenderManager* m_Pthis;
    vector<ImageRender*>* m_noTransVec;
    vector<FBXRender*>* m_fbxVec;
    vector<ImageRender*>* m_transVec;
    vector<Button*>* m_btnVec;
    vector<DebugRender*>* m_debugVec;

    ImVec2 m_winPos;
    LPDIRECT3DTEXTURE9 renderTargetTexture = nullptr;
public:
    static void Create();
    static RenderManager* GetInstance();
    static void Destroy();

public:
    void Register(ImageRender* ir);
    void Unregister(ImageRender* ir);
    void Register(FBXRender* fbxr);
    void Unregister(FBXRender* fbxr);
    void RegisterBtn(Button* btn);
    void UnregisterBtn(Button* btn);
    void RegisterDebug(DebugRender* db);
    void UnregisterDebug(DebugRender* db);

    void Initialize();
    void EditUpdate();
    void GameUpdate();
    void Release();

    ImVec2 GetWinPos();

    static int FrameCount;
};
```

5. 직접 구현한 2D 물리 처리 시스템 최적화

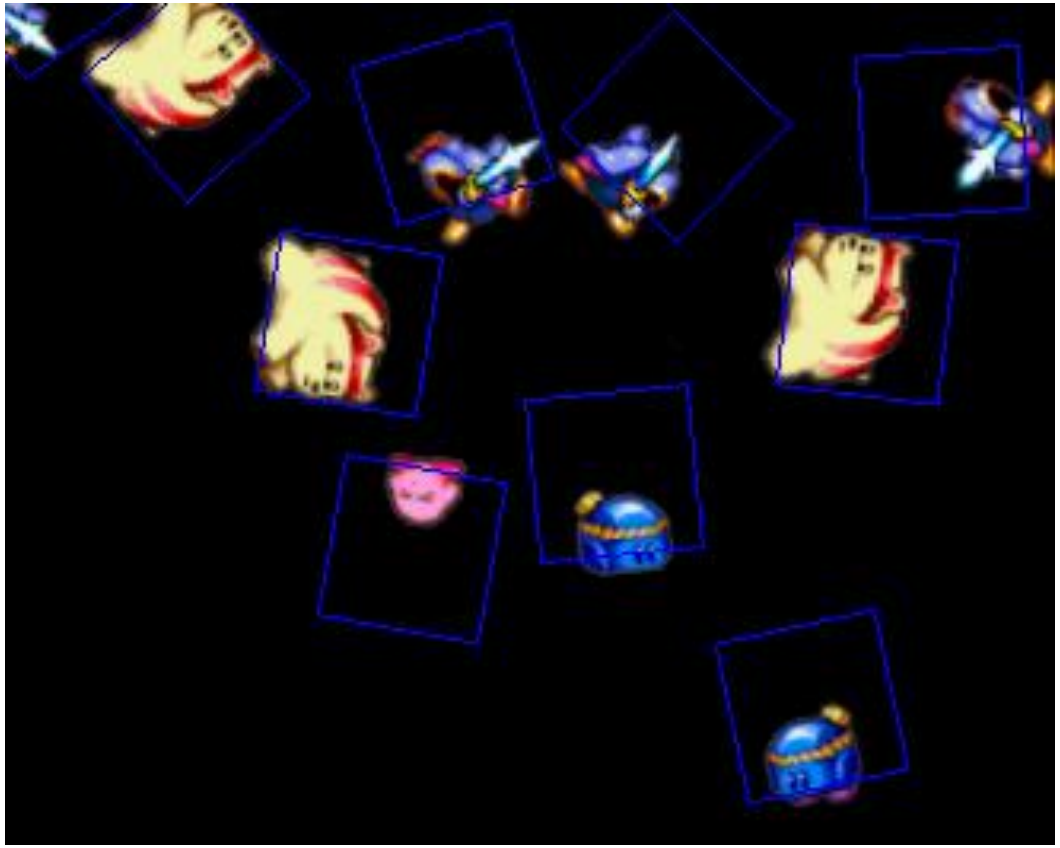


충돌체의 수가 많아질수록 충돌 검사의 횟수가 너무 많아지기 때문에, 이를 최적화하기 위해
2D공간을 재귀적으로 4분할하여 충돌 검사를
진행하는 다이나믹 쿼드 트리(공간 분할 알고리즘) 방식을
사용하였습니다.

2D공간을 4분할 하여 특정 영역의 충돌체 수가 일정 수를 넘어가면
다시 해당 영역을 4분할 하는 방식으로 동작합니다.

충돌 관련 이벤트는 충돌 시작, 충돌 중, 충돌 끝
3가지를 구현하였습니다.

6. Box2D 라이브러리를 활용한 2D 물리 시뮬레이션



직접 구현한 물리 처리 시스템보다
더 자연스럽고 고성능의 물리 처리를 위해,
물리 처리 로직을 Box2D라이브러리 기반으로 변경하였습니다.

이를 통해 물리 엔진의 복잡한 계산을
효과적으로 처리할 수 있게 되었으며,
충돌 반응 이벤트에 있어 높은 정확성과 안정성을
확보하였습니다.

7. ImGUI 라이브러리를 활용한 게임 툴 UI

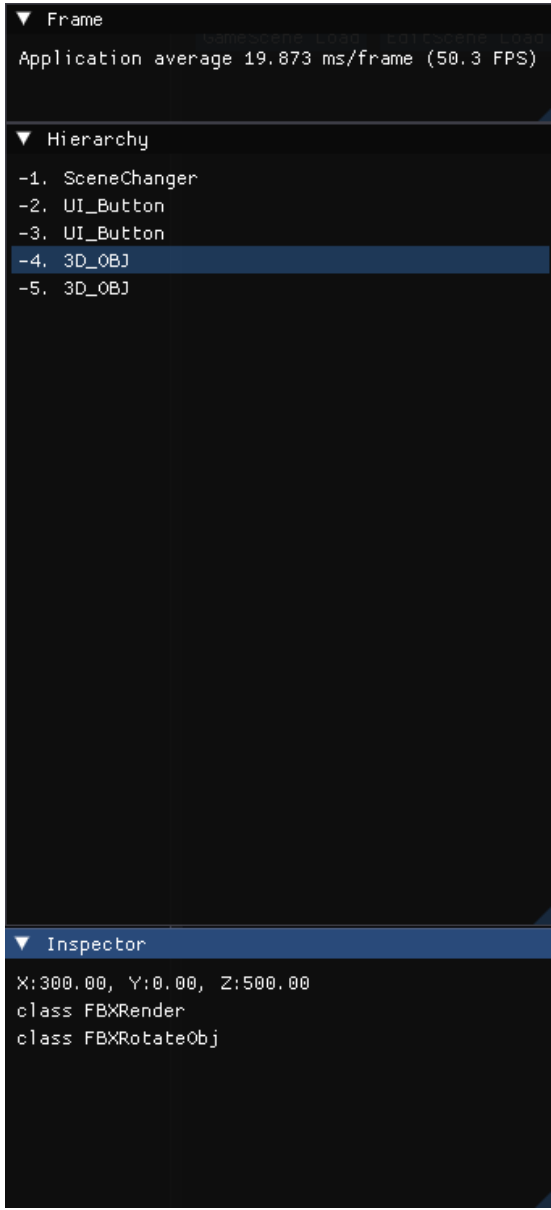
ImGUI라이브러리를 통하여
버튼, 문자열 입력과 같은 UI와

게임 오브젝트의 상태 및 정보,
보유 컴포넌트 목록,
현재 Scene의 이미지 출력 등

개발에 필요한 정보를
쉽게 확인할 수 있도록
하였습니다.

```
IMGUI_CHECKVERSION();
ImGui::CreateContext();
ImGuiIO& io = ImGui::GetIO();
io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard;
io.ConfigFlags |= ImGuiConfigFlags_NavEnableGamepad;
io.ConfigFlags |= ImGuiConfigFlags_DockingEnable;
io.ConfigFlags |= ImGuiConfigFlags_ViewportsEnable;
io.ConfigWindowsResizeFromEdges = false;

ImGui::StyleColorsDark();
ImGui_ImplWin32_Init(m_hWnd);
ImGui_ImplDX9_Init(m_pd3dDevice);
```

```
if (m_selected != m_objList->end() && !(*m_selected)->GetDestroy())
{
    ImGui::Begin("Inspector");

    sprintf_s(str, 64, "X:%.2f, Y:%.2f, Z:%.2f", (*m_selected)->Position().x, (*m_selected)->Position().y, (*m_selected)->Position().z);
    ImGui::Text(str);
    for (vector<Component*>::iterator itr = (*m_selected)->GetComponentVec()->begin(); itr != (*m_selected)->GetComponentVec()->end(); itr++)
    {
        const type_info& typeInfo = typeid(**itr);
        ImGui::Text(typeInfo.name());
    }

    ImGui::End();
}
```



8. FBXSDK를 활용한 FBX파싱과 렌더링 적용

FBXSDK를 활용하여,
우측에 정의된 서브메쉬와 모델 구조체에 맞게,
FbxTool클래스로
Fbx파일을 직접 DirectX9구조에 맞게 파싱하여
데이터를 가져온 후 이를 바탕으로 렌더링합니다.

```cpp
class FbxTool
{
public:
    FbxTool() : m_sdkManager(nullptr), m_scene(nullptr), converter(nullptr), textureCache(){}
    ~FbxTool() { Cleanup(); }

    bool Initialize();
    bool Load(const char* fileName, std::vector<Model>& outModels);

    std::vector<std::string> m_vertexInfo;
    std::vector<std::string> m_indexInfo;

    void CreateIndexBuffer(Model& model);
    void CreateVertexBuffer(Model& model);
private:
    FbxManager* m_sdkManager;
    FbxScene* m_scene;
    FbxGeometryConverter* converter;

    void ProcessNode(FbxNode* node, std::vector<Model>& outModels);
    void ProcessMesh(FbxMesh* mesh, Model& model);
    void LoadMaterial(FbxSurfaceMaterial* material, SubMesh& subMesh);
    void Cleanup();

    std::unordered_map<std::string, IDirect3DTexture9*> textureCache; // 텍스처 캐시 추가
};
```

```cpp
struct SubMesh {
    unsigned int startIndex; // 시작 인덱스
    unsigned int indexCount; // 인덱스 수
    unsigned int vertexCount; // 정점 수
    D3DCOLOR diffuseColor; // 색상
    std::vector<IDirect3DTexture9*> textures; // 텍스처 목록

    FLOAT uvScaleU;    // UV 스케일 U
    FLOAT uvScaleV;    // UV 스케일 V
    FLOAT uvOffsetU;   // UV 오프셋 U
    FLOAT uvOffsetV;   // UV 오프셋 V
};

//모델
struct Model
{
    std::vector<CUSTOMVERTEX> vertices;
    std::vector<unsigned int> indices;
    std::vector<SubMesh> subMeshes;  // 서브 메시 목록
    IDirect3DVertexBuffer9* vertexBuffer = nullptr;
    IDirect3DIndexBuffer9* indexBuffer = nullptr;
    int indexCount = 0;
    int vertexCount = 0;
    // 생성자
    Model() = default;

    // 소멸자: 버퍼 해제
    ~Model() {
        if (vertexBuffer) {
            vertexBuffer->Release();
            vertexBuffer = nullptr;
        }
        if (indexBuffer) {
            indexBuffer->Release();
            indexBuffer = nullptr;
        }
    }
};
```

9. 텍스처 리소스 관리 시스템-1

```
IDirect3DTexture9* TextureManager::GetTexture(const std::string& filepath) {
    // 맵에서 텍스처를 검색
    auto it = m_textureMap.find(filepath);
    if (it != m_textureMap.end()) {
        // 이미 존재하는 텍스처 반환
        return it->second;
    }

    // 새로운 텍스처 로드
    IDirect3DTexture9* texture = nullptr;
    if (SUCCEEDED(D3DXCreateTextureFromFileA(MainFrame::GetInstance()->GetDevice(), filepath.c_str(), &texture))) {
        m_textureMap[filepath] = texture; // 맵에 추가
        return texture;
    }

    return nullptr; // 로드 실패 시 nullptr 반환
}
```

여러 객체가 같은 텍스처를 사용할 때, 같은 텍스처를 여러 개 로드 하는 일을 방지하기 위해 만든 텍스처 매니저 입니다.

위 함수는 해당 매니저의 주된 기능으로,
주어진 파일 경로의 **이미 로드 된 텍스처가 있으면 해당 포인터를 반환**하고,
없으면 텍스처를 새로 로드 하여 해시 맵에 저장하고 반환합니다.

9. 텍스처 리소스 관리 시스템-2

```
//비동기 텍스처 로딩 함수
void TextureManager::GetTexture(const std::string& filepath, std::function<void(IDirect3DTexture9*)> func)
{
    // 맵에서 텍스처를 검색
    auto it = m_textureMap.find(filepath);
    if (it != m_textureMap.end()) {
        // 이미 존재하는 텍스처가 있다면 콜백 호출
        func(it->second);
        return;
    }

    // 새로운 텍스처 비동기 로드
    std::async(std::launch::async, [this, filepath, func]() {
        IDirect3DTexture9* texture = nullptr;
        if (SUCCEEDED(D3DXCreateTextureFromFileA(MainFrame::GetInstance()->GetDevice(), filepath.c_str(), &texture))) {
            this->m_textureMap[filepath] = texture; // 맵에 추가

            // 로드가 완료되면 콜백 호출
            func(texture);
        }
        else {
            // 로드 실패 시 nullptr
            func(nullptr);
        }
        });
```

콜백 함수를 등록해 **비동기 텍스처 로드**도 가능합니다.

```
ImageRender* ir = new ImageRender(nullptr);
ir->SetTrans(false);
AnimationManager::LoadTexture("Bitmaps\\obj\\land.bmp", ir);
block->AddComponent(ir);
```

(실제 사용 예시)

```
void AnimationManager::LoadTexture(const string& path, ImageRender* ir)
{
    char wpath[MAX_PATH] = { 0 };
    GetModuleFileName(NULL, wpath, MAX_PATH);
    //USES_CONVERSION;
    string executepath = wpath;
    executepath = executepath.substr(0, executepath.find_last_of("\\/"));
    string currentDirectory = executepath;
    string searchPath = currentDirectory + "\\" + path;
    TextureManager::GetInstance()->GetTexture(searchPath, bind(&ImageRender::LoadTextureCallback, ir, std::placeholders::_1));
}
```

StarLight 게임 소개

Unity 2022.3.8f1, C#

별빛을 모아 우주선을 업그레이드 하는 모바일 클리커 게임입니다.
주인공 캐릭터와 상호작용 할 수 있으며,
여러 스킬과 업그레이드를 통해 더 효율적으로 별빛을 모을 수 있습니다.

StarLight 게임 시스템-JsonLoader

```
public static class JsonHelper
{
    public static T[] FromJson<T>(string json)
    {
        Wrapper<T> wrapper = UnityEngine.JsonUtility.FromJson<Wrapper<T>>(json);
        return wrapper.Items;
    }

    public static string ToJson<T>(T[] array)
    {
        Wrapper<T> wrapper = new Wrapper<T>();
        wrapper.Items = array;
        return UnityEngine.JsonUtility.ToJson(wrapper);
    }


    [Serializable]
    private class Wrapper<T>
    {
        public T[] Items;
    }
}
```

게임에 필요한 각종 데이터를 Json파일로 저장, 로드 합니다.

StarLight 게임 시스템-UI

MainUI

SubUI

ElementUI

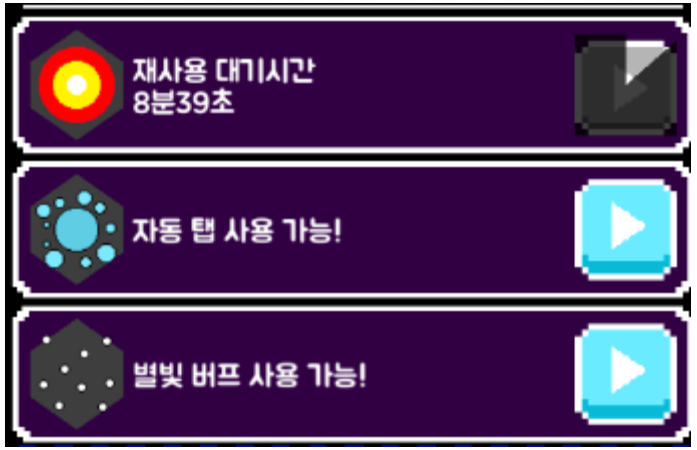
게임 내 UI 시스템은 전체적으로
MainUI – SubUI – ElementUI 순으로 계층 구조로 관리됩니다.

```
public void UpdateUIAll()
{
    UpdateStateUI();
    C_PopUpController.Instance.UpdateBtnText();
    m_cOption.UpdateText();
    m_cShop.UpdateUI();
    m_cChangeMode.UpdateText();
    for (int i = 0; i < m_arSubUI.Length; i++)
    {
        m_arSubUI[i].UpdateUIALL();
    }
    m_textAutoSave.text = C_BaseDataController.arCSTR[(int)m_cUserData.eLanguage].AutoSave;
}
```

MainUI에서 전체 게임 UI를 Update하는 부분

StarLight 게임 시스템-스킬

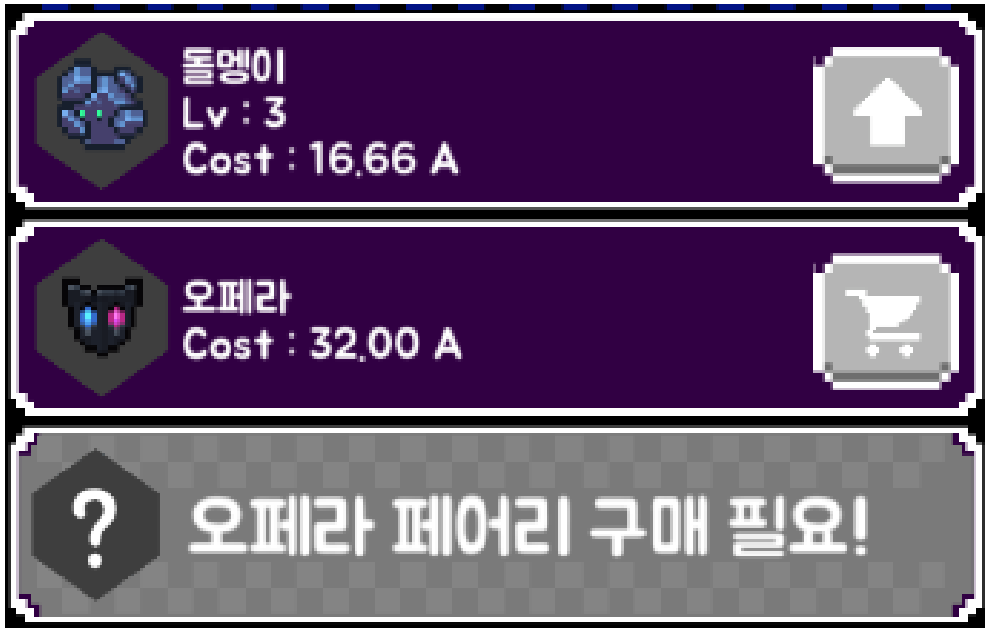
```
IEnumerator CoroutineCooldown_Init(E_Skill eSkill)
{
    WaitForSeconds loadUpdateTime = new WaitForSeconds(0.1f);

    while (!C_GameSceneController.IsInit)
    {
        yield return loadUpdateTime;
    }
    m_cSkillUIController.SetActiveBtnInteractable(eSkill, false);
    float fWaitTime = 1.0f;
    WaitForSeconds waitTime = new WaitForSeconds(fWaitTime);
    while (m_cUserData.arSkillCooldown[(int)eSkill] > 0)
    {
        m_cUserData.arSkillCooldown[(int)eSkill] -= (int)fWaitTime;

        if (C_MainUIController.Instance.IsUIActive() &&
            C_MainUIController.Instance.GetCurrentActiveSubUI() == E_SubUI.skill)
        {
            m_cSkillUIController.UpdateCooldownText(eSkill);
            m_cSkillUIController.SetCoolDownIMGFillAmount(eSkill,
                (float)m_cUserData.arSkillCooldown[(int)eSkill] / (float)m_arSkillData[(int)eSkill].nCooldownTime);
        }
        yield return waitTime;
    }
    m_cUserData.arSkillCooldown[(int)eSkill] = 0;

    if (C_MainUIController.Instance.IsUIActive() &&
            C_MainUIController.Instance.GetCurrentActiveSubUI() == E_SubUI.skill)
    {
        m_cSkillUIController.UpdateSkillElement(eSkill);
        m_cSkillUIController.SetActiveBtnInteractable(eSkill, true);
    }
}
```

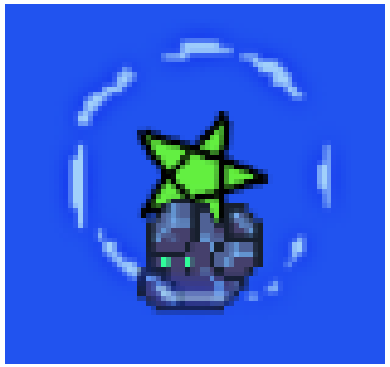


스킬의 **재사용 대기시간은 UserData로 저장**되어, 만약 게임 종료 시점에 대기시간 중인 스킬이 있다면 다음 게임이 시작될 때 바로 적용됩니다.

각 스킬들의 재사용 대기시간, 이름, 설명 등 기본적인 정보는 **Json파일**로 로드됩니다.

StarLight 게임 시스템-페어리

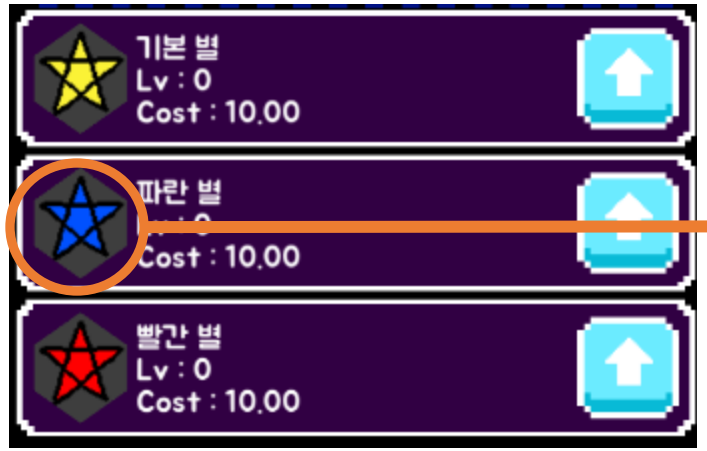




페어리(펫)을 구매하면
자동으로 일정 시간마다 별빛을 모아줍니다.

페어리에 대한 데이터 또한 Json파일로 로드되며,
이미 구입한 페어리는 업그레이드가 가능하고
다음 페어리의 구매 기능의 해금됩니다.

StarLight 게임 시스템-업그레이드

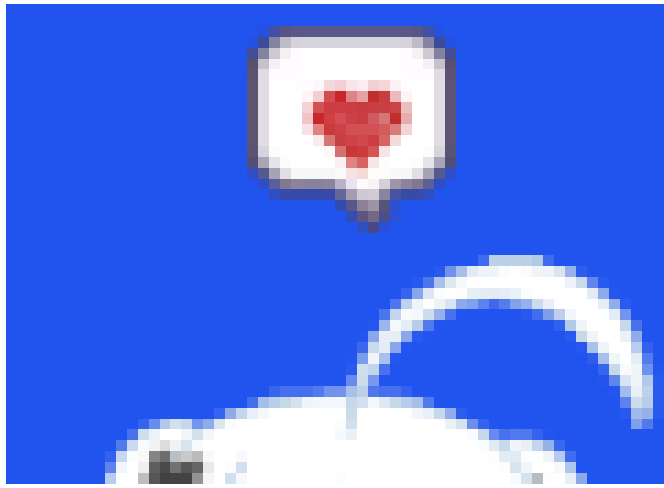


각종 업그레이드 기능 데이터도 Json파일로 로드되며,

좌측의 아이콘을 탭하면
캐릭터가 해당 업그레이드에 대한 설명을 해줍니다.

StarLight 게임 시스템-다이얼로그

Json파일로 저장된
캐릭터 대사



Dialogue Box에서는 직접 정의한 양식에 맞게 텍스트를 넣으면
해당 텍스트를 파싱하여
대화 단위 당 표정, 말풍선 종류, 특정 애니메이션의 재생 유무 등을 지정한 뒤
대화를 진행합니다.

기본적으로 입 모양이 뻥긋 거리는 애니메이션이 재생되며,
한 글자씩 출력됩니다.
문장 출력 중 터치 시 출력 중이던 대화 단위 문장이 한번에 출력됩니다.

StarLight 게임 시스템-세이브

```
public void Save()
{

    BinaryFormatter bf = new BinaryFormatter();
    MemoryStream stream = new MemoryStream();

    bf.Serialize(stream, m_cUserData);
    PlayerPrefs.SetString("UserData", Convert.ToBase64String(stream.GetBuffer()));
    stream.Close();
}
```

일정 시간마다 유저 데이터를 자동으로 저장합니다.
이때, PlayerPrefs를 통해 저장하며 데이터를 Base64로 인코딩합니다.

StarLight 게임 시스템-숫자변환

```
static private readonly string[] units = { "", "A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M", "N", "O", "P",
                                           "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z"};
static public readonly double MAX = 1286569735282830000000000000000000000000.00;

public string GetUnitText(double dData)
{
    string strResult = "";

    string[] arSplited = dData.ToString("N").Split(',');

    if (arSplited.Length >= 2)
    {
        strResult = string.Format("{0}.{1} {2}", arSplited[0], arSplited[1].Substring(0, 2), units[arSplited.Length - 1]);
    }
    else
    {
        strResult = arSplited[0];
    }

    return strResult;
}
```

클리커 게임 특성상 필요한 큰 숫자를 표현하는 기능입니다.
GetUnitText함수를 통해 큰 숫자를 천 단위로 축약하여 표현할 수 있습니다.
예를 들어 1,000은 1.00 A, 1,000,000은 1.00 B로 변환해 주는 역할을 합니다.

감사합니다